

Training Memory Query Rewriters Without Evidence Labels

Ronald Yu CSCI 6970 MS Course Project Fall 2026

Introduction

Assistants with long-term memory search a store built from past conversations, so they can answer questions about things said weeks or months ago. However, nearly all of them search using the user's current message, embedded as-is [1], which is often a bad search key [2], [3].

For example, imagine asking an assistant "should I bring a jacket to dinner tomorrow?" And the memories that answer it are "I get cold easily," from three months ago, and "dinner's on the rooftop patio," from yesterday. Neither is close to "jacket" in embedding space. And tuning the retriever does not fix that, so the query should change.

PPRO (2026) [4] does this by training a small model to rewrite the message before searching, and freezes the memory store, the encoder, and the answer model so the query is the only thing that changes; reporting gains on LoCoMo [5]. The problem however is how it trains. PPRO scores a rewrite by its overlap with annotated evidence turns, plus its similarity to a reference answer. Both are human annotations, while a deployed assistant has the user's history and nothing else. Thus, the method cannot be trained on the data it is meant to be used for, which the authors name as an open limitation.

This project trains a memory query rewriter with no evidence labels, and the training reward is built from the conversation itself. Three ways of building it are implemented and compared, which will be evaluated on LoCoMo's held-out test split. The result is a measurement: how much of a supervised rewriter's performance gain survives once the labels are gone.

The Gap

Training a model to rewrite queries is not new outside of memory. RRR [2] and CONQRR [3] did it for open-domain and conversational search years ago. Applying it to long-term memory is recent, and PPRO [4] is the strongest version, but it is supervised. Training a rewriter without labels is not new either. ConvSearch-R1 [6] and RL-QR [7] drop rewrite supervision, and AdaQR [8] builds a reward from conversation turns alone. All three of those search an external corpus, though.

Personal memory is a different kind of store. It is small enough to fit in a context window, there are no canonical documents in it, and the person asking the question is the same person who wrote everything in it. This project leans on the first of those. A user's whole history can be read offline while training, even though cost and latency mean only a few memories can be pulled once the system is actually running. That gap between training and serving is where a teacher signal can come from.

That is what this project does differently. PPRO showed that a trained rewriter helps memory retrieval, but it learns from human annotations. ConvSearch-R1, RL-QR, and AdaQR showed that a rewriter can be trained without annotations, but they search an external corpus, which is far too large to read offline the way a personal history can be. So this project takes the label-free idea into the memory setting, and uses the full conversation as the teacher that the annotations used to be. What it measures is how much retrieval performance is lost once the labels go away.

Research questions:

1. A rewriter trained on human-annotated evidence sets the bar. How close can a rewriter get to that when its training signal comes from the conversation alone?
2. Do the different training signals produce different queries, and do they fail on different questions? If the best of them could be picked per question, how much better would that be?
3. Does the answer change with the kind of question being asked, such as one whose answer sits in a single message versus one that needs two separate facts combined?
4. This project differs from PPRO in two ways at once: no labels, and a simpler training method. How much of the difference in results comes from each?

Contribution and Stakeholders

The project contributes three things:

- **A label-free training recipe.** A way to train a memory query rewriter with no annotated evidence, using a training signal built out of the conversation itself.
- **A head-to-head measurement.** Three of those signals compared against a rewriter trained on human annotations, split out by question type, so the cost of dropping labels comes out as an actual number.
- **A frozen harness.** Memory construction, the embedding model, the answer model, and the context budget all stay fixed, so someone can swap in a new training signal and compare it on equal terms.

Those map onto three groups:

- **Teams deploying memory systems on real user data.** They have conversation logs and no annotations, and they cannot produce annotations at scale. The first contribution runs on what they actually have, and the second tells them what they give up by not having labels.
- **Anyone running memory retrieval on-device or under a cost limit.** For them the rewriter has to be a small local model, not a frontier API call, and every configuration compared here is small enough to run locally.
- **Researchers working on memory retrieval.** They get the frozen harness plus a set of baselines all measured under the same conditions, so a later result can be compared to this one directly.

Approach

Two rules control the project:

- Gold evidence annotations never enter a label-free training loop. The supervised ceiling is the only thing that reads them, and it only reads them so it can be the ceiling. Everything else is trained without them, and all of it is scored against them at evaluation.
- Everything except the rewriter is frozen: memory construction, encoder, answer model, retrieval depth, context budget, and the training corpus. Any difference that shows up came from the query.

The teacher signal is the training reward, and it takes the place of the annotation-based score PPRO uses. The loop around it is kept simple on purpose: generate several different rewrites of the same question, score each one with the chosen reward, keep the best ones, and fine-tune the model on those. That is rejection sampling followed by

supervised fine-tuning, which is what Learning to Summarize from Human Feedback [9] did, except the score here is manufactured instead of coming from a human.

A few details matter here. Fine-tuning uses LoRA, which trains a small set of added weights instead of the whole model, so it fits on modest hardware [10]. The candidates are generated with randomized decoding so they do not all come out the same, because if they are all alike the reward has nothing to pick between [11]. Scoring happens offline, which is the only reason reading the full conversation is affordable. At serving time the rewriter never sees the conversation at all, just the user's message.

Five components, each testable alone:

- **Dataset.** The LoCoMo dataset will be used for training and evaluation.
- **Memory pipeline.** Splits a conversation into memories, embeds them with a frozen encoder, and stores them. It uses a published configuration, not an invented one, so the baseline is recognizable.
- **Retrieval and budget harness.** Runs a query, returns memories up to a fixed token budget, and scores the result.
- **Reward bank.** Three label-free scorers behind one interface, so swapping the training signal is a config change. This is the core of the project.
- **Candidate factory and training loop.** Generates the candidate rewrites, scores them, assembles the winners into a fine-tuning set, and runs the fine-tune.

The three label-free rewards, plus the ceiling they are measured against:

- **R1, full-context teacher.** A LoCoMo conversation fits in a modern context window, so the question can be answered twice: once from whatever the candidate query retrieved, and once from the whole conversation. The reward is how much those two answers agree. The full conversation is a free teacher, and it is only free because personal history is small. AdaQR [8] is the closest prior work, but it uses answer probability over a corpus, while this teacher gets to see the entire store.
- **R2, masked-turn prediction.** Hide a message from the conversation, then build a query out of the messages that came before it. The reward is how much the retrieved memories help the model predict the hidden message. There is no question here, no answer, and no labels, so the conversation ends up supervising itself. This is also the cheapest of the three.
- **R3, synthetic evidence.** Pick a message, generate a question that it answers, and keep the pair. That gives question-and-evidence pairs for free, and the closest prior work is RL-QR's synthetic queries [7]. It has a known weakness, which is exactly

why it is worth including: a question generated from a message tends to reuse that message's wording, and that is the easy case where a rewriter is needed least.

- **R0, supervised ceiling.** PPRO's reward has two parts: how much the retrieved memories overlap with the annotated evidence, and how close the resulting answer is to a reference answer. R0 keeps only the first part, so a query scores well when it brings back the memories the benchmark marks as needed. The second part is left out because it grades the wording of the answer rather than what the query retrieved. Apart from that, R0 is trained like the others, with the same procedure, budget, and base model. It is not a competitor; every label-free result gets reported as a fraction of it.

All four rewards train on the same corpus, so the reward is the only thing that changes between them. What each one takes out of that corpus is different, though:

- **R2 and R3 need the conversations and nothing else.** R2 hides a message that is already sitting there, and R3 writes its own questions from messages that are already sitting there.
- **R1 also needs questions asked over those conversations.** It scores a candidate by how well the retrieved memories support the answer, so something has to do the asking. The answers themselves are not needed, since the full-conversation teacher produces those.
- **R0 also needs the evidence annotations,** and it is the only one that ever reads them.

So the corpus has to be an annotated one, even though three of the four rewards never touch the annotations. That is how the first rule stays intact while the data stays constant. The label-free loops never see a label, and the comparison against the ceiling only moves one thing instead of two.

What this project does not do:

- **Memory construction.** How memories are extracted, summarized, and indexed is frozen. Only the query changes.
- **Reranking.** Left out so the path from query to retrieved set stays unbroken.
- **Retriever training.** The encoder is frozen. The intervention is the query, never the index.
- **Reinforcement learning.** PPRO trains with reinforcement learning. This project uses the simpler generate-score-keep-fine-tune loop instead, which fits the hardware and is easier to debug. That difference gets reported as a result of its own, and it is research question 4.

- **Breaking one question into several queries.** It would be interesting, but it is a whole second contribution and this project already has one. It goes in future work.

Evaluation

The primary metric is recall of the annotated evidence under a fixed token budget: of the memories the benchmark marks as needed to answer a question, how many did this query actually bring back, with the retrieved set capped at a fixed number of tokens.

It is the primary metric because it measures the thing the query actually controls, and because everything downstream depends on it. If the needed memories never come back, no answer model can make up for that. The cap is what keeps the measurement honest. Without it, a query that drags back the entire conversation would score perfectly while being useless in practice. Results are reported as a curve over several budget sizes instead of one number, because a method that only wins when the budget is generous is a different story from one that wins everywhere, and real deployments differ a lot in how much context they can afford.

Two secondary metrics, for the things recall does not cover:

- **Rank of the first needed memory.** Two queries can bring back the same memories, but one puts the needed memory first and the other buries it near the bottom. That matters any time the list gets cut off, and answer models tend to use early context more reliably than late context.
- **How much of the token budget actually gets used.** A query that hits the same recall on half the budget is cheaper to serve and leaves room for the rest of the prompt. It is also an early warning: if a query starts drifting toward retrieving everything, this is where it shows up first.

Every number is reported per question category rather than as one average. LoCoMo labels each question by the kind of reasoning it needs:

- **Single-hop.** The answer sits in one message.
- **Multi-hop.** Two or more messages have to be combined.
- **Temporal.** About when something happened, or in what order.
- **Open-domain.** Needs outside knowledge on top of the conversation.

These are not equally hard, and rewriting should not help them equally. Single-hop questions often work fine with the raw message already, while multi-hop and temporal

questions are where a rewrite has the most to add. So one average can hide a big multi-hop gain behind a single-hop score that was already maxed out, or make a method that only helps the easy category look like a general improvement. Every configuration is also run more than once with different random seeds, so a small gap between two methods does not get read as a real one.

This follows a broader argument in the evaluation literature, which is that generated text should be judged by what it accomplishes and not by how much it looks like a reference [12]. That is also why a rewrite never gets scored by its similarity to a reference answer. What matters about a rewrite is what it retrieves, not how it reads.

The comparison runs along a ladder of six configurations, all evaluated on the held-out test split, with memory construction, encoder, answer model, retrieval depth, and budget frozen at every step:

0. Raw user message, embedded as-is. The status quo in nearly every memory system.
1. Prompted rewrite, small model. Rewriting with no training.
2. Trained on R1. Contribution.
3. Trained on R2. Contribution.
4. Trained on R3. Contribution.
5. Trained on R0, gold labels. Supervised ceiling.

Schedule

- **Phase 0, feasibility (early September).** The go/no-go check under Risks and Concerns. No training code gets written until it passes.
- **Phase 1, baselines (September).** Memory pipeline, retrieval and budget harness, and ladder steps 0 and 1 measured and frozen.
- **Phase 2, rewards and first training runs (October).** All three rewards implemented behind one interface, candidate generation and scoring working at scale, and the first trained rewriters.
- **Phase 3, full comparison (early November).** Remaining trained steps, the supervised ceiling, repeat runs with different seeds, and the per-category breakdown.
- **Phase 4, analysis and writing (mid to late November).** Research questions 2 through 4, then the report, with the remaining slack held here.

Risks and Concerns

Three things could actually stop the project. The first is a go/no-go check, resolved in Phase 0 before any training code exists:

- **No headroom over the raw message.** If the raw message already brings back the needed memories most of the time, there is nothing left for a rewriter to fix. The check is an oracle query built straight from the annotated evidence, which is the best any query could possibly do here.
- **Compute.** The GPU has 16GB, so training uses a roughly 10B-parameter model with LoRA and reduced-precision weights. Anything larger is used for inference only and never trained.
- **The training split is small.** It comes from a single conversation, which is thin, so the absolute numbers may come out modest. Every reward trains on the same corpus, so the comparison against the ceiling still holds. If it turns out to be too thin to train anything at all, the label-free rewards can fall back to an unlabeled multi-session corpus, which costs them that shared corpus. The test split stays held out either way.

Deliverables

- A working memory query-rewriting harness, five components, reproducible from a config file and a seed.
- A reward bank with three label-free signals plus the supervised ceiling.
- A training pipeline covering candidate generation, scoring, training-set assembly, and fine-tuning.
- Budget curves for all six ladder steps, reported per question category, with repeat runs.
- Analysis: how the three rewards compare, which questions each one fails on, what the best-of-three ceiling looks like, and how different the rewrites actually are from each other.
- A written final report.

Breadth and Mastery

NLP and GenAI

The project sits on top of the core topics of the course:

- **Word vectors and embedding spaces.** Retrieving memories is nearest-neighbor search in an embedding space, so what embeddings do and do not capture is the mechanism the whole project rests on. It is also where the main design decision comes from: a rewrite gets judged by what it retrieves.
- **Sequence-to-sequence models and attention.** Rewriting a question into a search query is conditional generation, the same input-to-output framing as translation.
- **Self-attention and transformers.** The rewriter is a decoder-only transformer. The embedding model is bidirectional and has a hard input length limit, which is part of why searching with the raw message fails in the first place.
- **Pretraining and transfer learning.** The training method is adapting a pretrained model to a narrow task with LoRA [10], and R2 is basically a pretraining-style self-supervised objective being used as a reward.
- **Prompting and learning from feedback.** Ladder step 1 is a prompted baseline, and the training loop is the Learning to Summarize from Human Feedback method [9] with a manufactured score standing in for human preference.
- **Natural language generation.** Candidate generation leans on the decoding literature [11], because candidates that all look alike leave the reward nothing to pick from. Evaluation follows the same literature's argument about judging generated text by what it accomplishes [12].

Rest of MS Studies

- **Machine learning.** Reward design, keeping an evaluation set completely out of training, ablations that move one variable at a time, and reporting seed-to-seed variance as part of the result.
- **Deep learning.** Fine-tuning a pretrained model under a hard memory ceiling: adapter-based training, reduced-precision weights, and the trade-off between how many candidates get generated and how fast the loop runs.
- **Information retrieval.** Dense retrieval, recall and rank metrics, and evaluation under a fixed budget.
- **Software engineering.** A five-component harness driven by config files and reproducible from a seed, with caching in the places where it decides whether something is feasible at all.

- **Data engineering.** Building on an existing dataset.

References

- [1] Chhikara, P., et al. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413, 2025. <https://arxiv.org/abs/2504.19413>
- [2] Ma, X., et al. Query Rewriting for Retrieval-Augmented Large Language Models (RRR). arXiv:2305.14283, 2023. <https://arxiv.org/abs/2305.14283>
- [3] Wu, Z., et al. CONQRR: Conversational Query Rewriting for Retrieval with Reinforcement Learning. Proceedings of EMNLP 2022. <https://aclanthology.org/2022.emnlp-main.679.pdf>
- [4] Jiang, Z., et al. Learning User-Aware Recall: Personalized Retrieval in Long-Term Conversational Memory (PPRO). arXiv:2607.00017, 2026. <https://arxiv.org/abs/2607.00017>
- [5] Maharana, A., et al. Evaluating Very Long-Term Conversational Memory of LLM Agents (LoCoMo). arXiv:2402.17753, 2024. <https://arxiv.org/abs/2402.17753>
- [6] Zhu, C., et al. ConvSearch-R1: Enhancing Query Reformulation for Conversational Search with Reasoning via Reinforcement Learning. arXiv:2505.15776, 2025. <https://arxiv.org/abs/2505.15776>
- [7] Cha, S., et al. Annotation-Free Reinforcement Learning Query Rewriting via Verifiable Search Reward (RL-QR). arXiv:2507.23242, 2025. <https://arxiv.org/abs/2507.23242>
- [8] Zhang, T., et al. Adaptive Query Rewriting: Aligning Rewriters through Marginal Probability of Conversational Answers (AdaQR). arXiv:2406.10991, 2024. <https://arxiv.org/abs/2406.10991>
- [9] Stiennon, N., et al. Learning to Summarize from Human Feedback. arXiv:2009.01325, 2020. <https://arxiv.org/abs/2009.01325>
- [10] Hu, E., et al. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685, 2021. <https://arxiv.org/abs/2106.09685>
- [11] Holtzman, A., et al. The Curious Case of Neural Text Degeneration. arXiv:1904.09751, 2019. <https://arxiv.org/abs/1904.09751>

[12] Liu, C.-W., et al. How NOT To Evaluate Your Dialogue System: An Empirical Study of Unsupervised Evaluation Metrics for Dialogue Response Generation. arXiv:1603.08023, 2016. <https://arxiv.org/abs/1603.08023>